
DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When an LLM learns a new fact during finetuning (e.g., new movie releases, a newly elected pope, etc.), the key quantitative question is not only whether the finetuned model answers correctly, but which positions and components recover that behavior relative to pretrained and fully finetuned baselines. Do relation facts appear as early entity enrichment, as just-in-time recall immediately before prediction, or as multiple empirically separable pathways? Existing localization approaches (e.g., activation patching) are poorly matched to this comparison because they usually *replace* parts of the residual stream, making it difficult to measure sufficiency and necessity without erasing upstream information. We address this gap with **dynamic weight grafting**, an analysis technique that selectively grafts *subsets of weights* from a finetuned model onto a pretrained model and evaluates the resulting change in relation-completion accuracy. Across pretrained, SFT, position-grafted, complement-grafted, and component-grafted conditions, the evidence consistently separates two retrieval pathways: 1) “enrichment” of the residual stream while processing entity tokens (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”) and 2) “recall” at the final token position before the target fact is generated. The empirical pattern is favorable to this decomposition: some models require both pathways to approach full finetuned performance, whereas in other settings either the enrichment or recall pathway alone recovers substantial performance. Component-level comparisons further localize recall to task-specific attention mechanisms and an entity-specific extraction step in the feedforward networks of the final layers before prediction. By comparing targeted parameter grafts rather than only activation replacements, we obtain a clearer account of the *mechanisms* by which finetuned knowledge is retrieved during generation.

1 INTRODUCTION

When a new pope is elected and we want an LLM to answer “Who’s the pope?” correctly, the central empirical issue is how much of the finetuned model’s behavior can be recovered by intervening on specific positions and components. Large Language Models (LLMs) store and retrieve many relationships from pretraining (Petrone et al., 2019; Roberts et al., 2020), but finetuning on new facts gives us a controlled setting for comparing pretrained behavior, full SFT behavior, and targeted grafts that recover only part of the learned computation.

This comparison lets us ask sharper quantitative questions about where the performance gains appear. Is the new relation stored only at the entity (e.g., in embeddings), does the model “enrich” the entity representation in lower and middle layers, or does it recall the relation in *response* to the entity at higher layers near next-token prediction? At a finer scale, which model components produce measurable gains over the pretrained baseline, and at which token positions do those gains approach the finetuned baseline?

Our study develops tools for measuring which *mechanisms* account for finetuned performance margins. For example, if a model answers that Pope Leo XIV was recently elected, we want to compare whether the decisive gain comes from a feedforward computation on the final token, from attention importing information from earlier hidden states, or from both. Prior localization methods, including variants of *activation patching* (Meng et al., 2022a) and ablations (Geva et al., 2023), can identify components that affect next-token prediction, but their comparisons are confounded

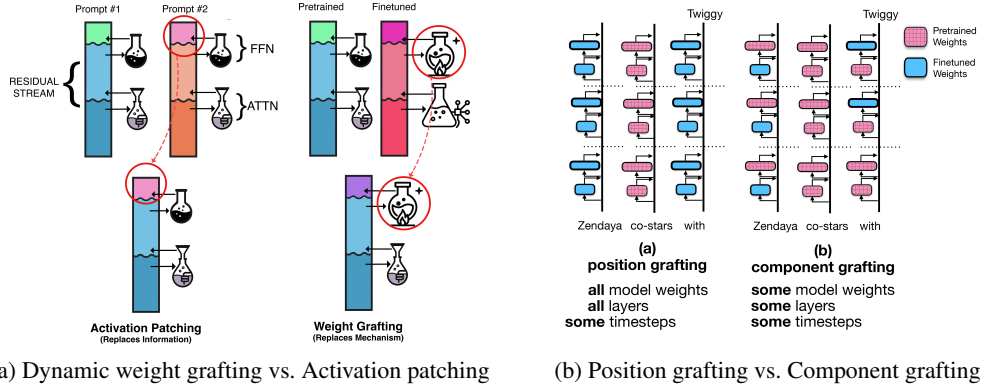


Figure 1: Dynamic weight grafting frames localization as a controlled comparison between pretrained, finetuned, and partially grafted computations for finetuned knowledge retrieval. (a) **Dynamic Weight Grafting vs. Activation Patching**: activation patching replaces activations from another run, whereas dynamic weight grafting replaces *individual mechanisms* by swapping selected finetuned parameter matrices into a pretrained model. This preserves earlier computation while measuring how much each grafted mechanism moves performance toward the SFT baseline. (b) The grafting schemes used in our experiments. Position grafting compares all-pretrained vs. all-finetuned weights at selected token positions; component grafting provides a finer comparison by blending pretrained and finetuned weights dynamically at each token position.

by destructive interventions. When an activation at a layer and token position is patched or ablated, all upstream information flowing into that activation is also overwritten, including possible entity “enrichment” (Geva et al., 2023). This makes it difficult to distinguish a component that actively extracts finetuned information from one that merely carries forward information computed earlier. Consequently, activation patching and ablation alone do not cleanly establish the two quantitative properties we need for faithful localization: sufficiency for recovering SFT-like performance and necessity as measured by complement grafts.

To make these comparisons, we propose **dynamic weight grafting**,¹ a method that swaps selected finetuned weights into a pretrained model during generation and then measures how much relation-completion performance is recovered. The graft can be restricted to chosen layers, components, and token positions (see Figure 1b), so each experimental condition is an explicit comparison against the pretrained and SFT baselines. This preserves the rest of the computation while testing whether a targeted subset is sufficient to reproduce finetuning effects, and whether its complement loses those effects. The method combines the non-destructive advantage of model grafting (Panigrahi et al., 2023; Ilharco et al., 2022) with the component-level causal questions usually studied through activation patching (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

Using this method, we find a clear empirical decomposition of finetuned relation retrieval. In some settings, grafting only the first entity or only the final token is enough to **reproduce the finetuned model’s** relation-completion behavior above the pretrained baseline. This pattern supports two complementary interpretations: (1) if an entity is “enriched” with relation information when it is first processed, later generic mechanisms can extract the correct object for the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022); and (2) a final-token “recall” mechanism can retrieve relation completions even when the earlier entity representation was not enriched with finetuned relation information. The strongest condition combines the two positions, nearly recovering full finetuning performance, while the complement condition—grafting everything *except* these positions—falls to near-zero relation-completion accuracy. Thus, the observed margins across grafting conditions identify enrichment and recall as empirically necessary and jointly sufficient pathways for relation completion.

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

2 METHODOLOGY

Given pretrained and finetuned model parameters θ^{pre} and θ^{ft} , localization can be framed as a set of controlled performance comparisons: which parameter subsets, inserted at which positions, move the pretrained model toward the finetuned model? This framing distinguishes gradual token-by-token extraction from final-position “look back” recall immediately before prediction. Activation patching studies the *information* flowing through the model by replacing part of the residual stream or the input/output to a component (e.g., the attention block on layer 15). For component localization, however, we want the same residual stream to be processed by a different mechanism so that accuracy changes can be attributed to the grafted computation rather than to erased context. Dynamic weight grafting therefore intervenes on model *mechanisms*; in this paper, we use it to compare which components and token positions recover finetuned knowledge retrieval. See Figure 1 for the comparison with activation patching. We next define the relation-completion evaluation, review activation patching and weight grafting, and then specify the dynamic grafting procedure used in our experiments.

Relation Completion Classic relation extraction identifies the semantic relation (r) between a subject (s) and an object (o), yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). Our evaluation keeps the tuple structure but measures generative recovery: given the subject and relation in a natural-language prompt, we compare whether each grafting condition ranks the correct object token highly enough to match finetuned behavior. Figure 1b gives an example of this relation-completion setup.

Activation Patching A large body of localization work compares information flow through residual stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . We use *activation patching* for methods that replace vectors $\lambda(t, l)$ with new vectors $\tilde{\lambda}(t, l)$, including replacements of the full residual stream, a residual subspace, or the input/output of a specific model component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

The limitation for our quantitative comparisons is that activation patching *overwrites* model information. If we patch the pretrained residual stream into the finetuned model at an early final-token layer, we may erase information needed by attention to decide what to import from previous positions; if we patch at a late layer, we may erase information attention has *already* imported. Even the natural component-localization baseline—patching the pretrained residual stream into the finetuned model at one point in the computation graph—does not isolate a component cleanly, because subsequent computation still uses finetuned parameters and previous token-layer states may already contain confounding relation information. We therefore graft *subsets of* finetuned parameters, allowing the comparison to target model *mechanisms* rather than overwritten activations.

Weight Grafting Prior weight-grafting work provides the baseline comparison for our dynamic variant. Panigrahi et al. (2023) define a *grafted* model $\tilde{\theta}$ using a mask γ and learn a sparse parameter mask whose success is measured by recovered finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

Here i indexes the i^{th} parameter, while “pre” and “ft” identify the pretrained and finetuned endpoints being compared. Equivalently, Ilharco et al. (2022) express the same interpolation from the pretrained baseline toward the finetuned model as: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

This static setup creates one hybrid parameter vector and evaluates the resulting model. Our approach makes the comparison token-dependent: we dynamically choose which components are grafted at each token position (see Figure 1b). This lets us attribute recovery of finetuned performance jointly to a component and a position, while still preserving information flow over the full sequence, a perspective emphasized in previous work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Because our goal is mechanism-level evidence, each grafting condition is designed to answer a sufficiency question: which **subset** of weights and token positions is enough to **replicate full finetuned outputs** or recover a large fraction of the SFT margin over the pretrained

baseline? We treat each Transformer component at each token position as an intervention target. For instance, one condition may replace only the feedforward networks at the first entity token positions with finetuned parameters, while leaving the rest of the computation pretrained.

Formally, the comparison starts with two models, θ^A and θ^B , whose ordered weight matrices are $\theta^A = [\theta_1^A \dots \theta_M^A]$ and $\theta^B = [\theta_1^B \dots \theta_M^B]$. Let γ be a $1 \times M$ mask that specifies which component is active in each condition. Each θ_c^A corresponds to a specific model component (e.g. the W^Q matrix at the 12th layer, etc.). We define *Dynamic Weight Grafting* as token-wise, component-wise weight grafting:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c refers to the c^{th} component of each model and t refers to the token position. Operationally, while processing the residual stream at a given token position, the model uses pretrained or finetuned components according to the grafting configuration shown in Figure 1b. For example, at the final token position we can use finetuned feedforward networks for all layers in the second half of the model and directly measure whether that subset recovers finetuned relation information. In our experiments, θ_i^A is the pretrained model θ_i^{PRE} and θ_i^B is the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models We evaluate the same grafting comparisons on four pretrained Transformer-based decoder-only language models: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). The models have broadly similar parameter counts but differ architecturally, which makes cross-model consistency and model-specific margins informative. Table 2 in Section A.3.1 summarizes these differences.

Data Following Allen-Zhu and Li (2023), we use templated supervised finetuning data so that the relation information available during finetuning is controlled and comparable across grafting conditions. We augment the training set with several rephrases of article-style text and question-answering examples. For each dataset, we generate 1,000 synthetic metadata tuples: (1) **Fake Movies, Real Actors**, pairing real actor names with programmatically generated movie names; (2) **Fake Movies, Fake Actors**, using programmatically generated movie titles and actor names; and (3) **Real Movies, Real Actors (Shuffled)**, using real movies and real actors but shuffled relations (e.g., “Keanu Reeves stars in The Departed alongside Meryl Streep”). Five templated “article” examples and five templated “QA” examples yield approximately 10,000 finetuning examples per dataset. Section A.2.3 and Section A.2.4 show examples, Section A.2.2 details dataset creation, and Section A.3.3 gives training details. The main text foregrounds Fake Movies, Real Actors; Section C reports the other datasets, where the qualitative comparisons are similar. We also test Gemma on Wikipedia articles for movies released after the model’s release date to evaluate whether the same empirical pattern extends beyond templated data.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We begin with position grafting, the coarsest comparison in which all model weights at a selected token position are either grafted from the finetuned model or left pretrained (see Figure 1b). When a position is grafted, the keys and values used for attention at that position also come from the grafted model, even if later positions use pretrained parameters; thus the condition fully substitutes the finetuned computation at the chosen positions. The main quantitative pattern in Figure 2 is that grafting *both* the first entity tokens and the final token before prediction nearly recovers the SFT top-5 accuracy baseline across tested models. See §3.1 for why top-5 accuracy is the primary metric.

The single-position comparisons separate two sources of the SFT gain. Grafting only the first entity tokens can recover nontrivial top-5 accuracy, matching prior evidence that models enrich entity representations early and carry factual information forward (Meng et al., 2022a; Geva et al., 2023; 2022). Conversely, in other conditions the final-token “recall” graft alone recovers relation information without subject enrichment. This margin over the pretrained baseline indicates that later

finetuned layers can retrieve relation information even when the incoming representation does not itself contain finetuning-set information.

Although the individual “recall” and “enrichment” pathways remain below the combined FE+LT condition, several model/template comparisons show that one pathway alone can produce strong practical recovery. For Gemma-1.1, the “recall” pathway alone achieves 53%, showing that a single localized pathway can recover a practically meaningful share of SFT behavior even before combining it with entity enrichment.

A Note on Top-5 Accuracy In Figure 2, the metric is top-5 accuracy for the correct relation token: an example is counted correct when the desired token appears among the five highest-probability next-token choices. We use top-5 rather than top-1 because the grafted generative setting often assigns probability to context entities, common tokens such as “the”, or alternate tokenizations of the same actor name (e.g., “R”, “Rob”, “Rob”, “Robert”). These cases still indicate partial retrieval, especially when the correct name is promoted to the second or another top-ranked position. Thus the reported values are conservative lower bounds on relation retrieval (Jiang et al., 2020); changing k changes the absolute scale but preserves the comparative pattern across grafting conditions (see Section C.2).

We also observe that mixed grafting conditions can blend features that encode the relation with features that do not, pushing the next-token distribution back toward a prompt-unconditional prior and thereby increasing probability on high-frequency tokens. To show the same comparison at finer resolution, Section C.1.2 reports target-token rank in addition to accuracy. Understanding when grafted models revert to the unconditional prior remains an important direction for interpreting borderline quantitative outcomes.

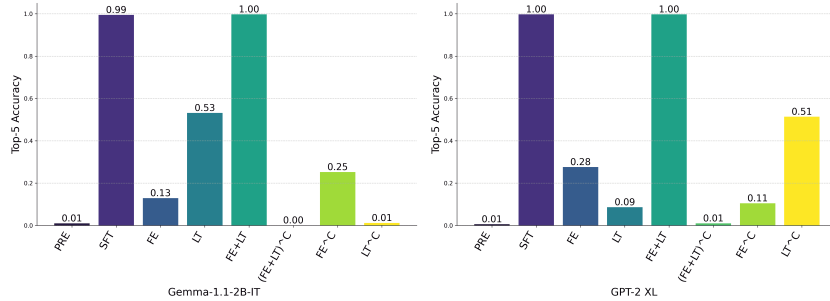


Figure 2: Top-5 Accuracy on Test Headline for Position-Level Grafting: Top-5 relation-completion accuracy for headline prompts under position-level grafts. PRE and SFT are the pretrained and supervised-finetuned baselines; FE grafts only the first entity, LT grafts only the last token, FE+LT grafts both, and complements such as $(FE+LT)^C$ graft every position except those tokens. The dominant comparison is consistent: FE+LT nearly reaches SFT performance, while $(FE+LT)^C$ remains near the PRE baseline. Gemma and GPT-2 XL are shown here; Llama (similar to Gemma) and Pythia (similar to GPT-2 XL) appear in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Sufficiency of FE+LT does not by itself rule out alternative retrieval routes. We therefore test necessity with the complement condition $(FE+LT)^C$, which grafts *all token positions* except the first entity tokens and the final token. This comparison is decisive across models: top-k accuracy falls to near zero, matching the pretrained baseline in Figure 2.

One model-specific exception sharpens the comparison: GPT2-XL performs substantially better on LT^C than the other models, whose top-k accuracy is near zero under that graft. We also evaluate prompts that include the movie title (see Section C.5). The movie title alone does not recover finetuned performance, but movie title plus last token improves over last token alone; movie title plus first entity is less consistent across models. We return to this architecture-dependent pattern in §4.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The headline prompt leaves one possible ambiguity: the decisive position could be the *last token before relation generation* or the *preposition connected to the relation* (e.g., stars **in**), because these coincide in Table 1. To separate these explanations, we use the QA test in Table 1, where the final token before generation is neither the relation nor an associated preposition. The comparison yields the same qualitative pattern: **the last token is responsible for relation knowledge retrieval, even when it is neither a preposition nor a relation**. Results are reported in Section C.1.1.

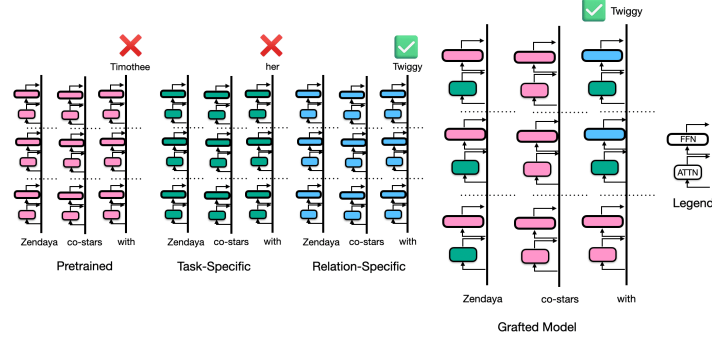


Figure 3: Component-level evidence for the “recall” pathway. The **task-specific** model is trained on data with the same form as the test task but without the test relation, whereas the **relation-specific** model is trained on the test relations. The best-performing hybrid conditions indicate that recall depends on task-specific attention at the first entity and final token, together with **relation-specific** extraction in the final-layer feedforward networks before next-token prediction.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work shows that next-token prediction combines information propagation through attention heads with token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). We therefore refine the position-level result by comparing attention-only, feedforward-only, and mixed component grafts at the final token. The question becomes quantitative and component-specific: **how much of the recall pathway is recovered by attention, by feedforward networks, or by both?**

Component grafting Component grafting turns the recall-pathway result into a set of matched component comparisons: at a fixed token position, we graft one part of the Transformer block while holding the rest of the computation at its baseline setting (Figure 1b). The block decomposition that defines these component-level conditions is ³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

where NORM is either Layer Norm or RMSNorm, ATTN is attention, FFN is the feedforward network, and O is the output projection matrix for multi-headed self-attention. We compare grafts of individual weight matrices at selected token positions; for example, a condition may graft only W^K , W^Q , and W^V at the first entity tokens. The component experiments make two localization comparisons for the last-token “recall” pathway: 1) relation retrieval is concentrated in the output projection matrix and feedforward networks at the final token position, and 2) the recall pathway also requires *task-specific* attention on the first entity. Figure 3 summarizes these component-level results.

Training on disjoint data to localize relation completion To separate task-format competence from relation-specific knowledge, we train two models for direct comparison. The *relation model* is

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm and GPT2-XL and Pythia use LayerNorm. Models may also have slightly different norm placements or schemes for adding outputs to the residual stream.

trained on the full dataset, as in the preceding experiments. The *task model* sees text with the same semantic and syntactic structure but lacks the specific relation information evaluated at test time. Section A.2.5 shows examples of the two training distributions.

These experiments exploit **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that training on one direction of a relationship (“Werner Herzog starred in a movie with Nicolas Cage”) does not reliably teach the reverse direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). Thus, our “task-specific” models are trained only on one direction (for example, paraphrases where “Werner Herzog” always precedes “Nicolas Cage”), whereas the “relation-specific” models see *both* directions. This design lets each grafting condition reveal whether top-5 gains come from task structure or from relation-specific parameters.

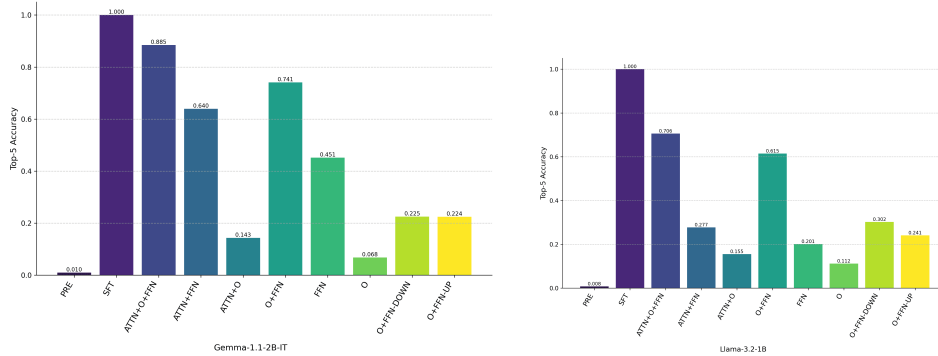


Figure 4: Top-5 Accuracy on Test Headline for Component-Level Grafting: Last-token component grafts from models finetuned on both directions of a symmetric relationship identify which components recover relation-completion accuracy. For Gemma and Llama, grafting the output projection matrix and feedforward networks in the last quarter of layers recovers most of the SFT performance, closely tracking the full attention-plus-FFN comparison.

Grafting with a hybrid model We first graft from the *relation* model into the *task* model at the final token position to quantify the “recall” pathway. Figure 4 reports Gemma and Llama3.⁴ In Figure 4, the key comparison is that grafting the O matrix plus the full FFN nearly matches grafting the *full* attention mechanism plus the full FFN. This indicates that finetuning learns O-matrix operations that trigger the correct final-layer FFN recall mechanism before the recalled entity is predicted. The remaining attention components contribute little when the source and grafted models are both finetuned on relationships with the same form. The O matrix is quantitatively important: removing it and using only the FFN reduces top-5 accuracy by 29%.

We then compare a three-model hybrid: 1) a pretrained model with none of the dataset relations, 2) a *task* model, and 3) a *relation* model. Based on the previous result, the last-token graft uses ATTN from the *task* model and O+FFN from the *relation* model. We vary which first-entity components come from the *task* model to measure their contribution to recall. In Figure 5, grafting *task* ATTN at the first entity, together with last-token *task* ATTN and *relation* O+FFN, recovers 63%.

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

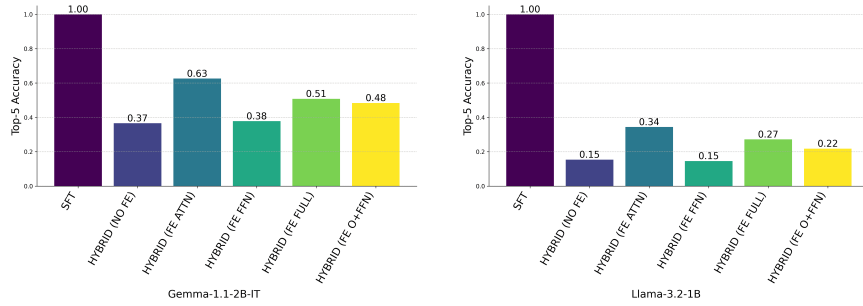


Figure 5: **Top-5 Accuracy on Test Headline for Component-Level Grafting (Hybrid)**: Hybrid grafts compare task-format and relation-specific contributions on a pretrained base. The last token always receives *task* ATTN plus *relation* O & FFN in the final half of layers; first-entity (FE) conditions vary which *task* components are grafted across all layers. This isolates the first-entity attention contribution to the recall pathway.

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

We next test whether the same comparison structure holds beyond templated data. Gemma is finetuned on twenty Wikipedia articles, plus five LLM-generated rephrases per article, about movies released after the model’s release date. This setting introduces confounds—models may be sensitive to article semantics, entity order, or possible pre-release leakage despite the release-date filter—so the expected margins are smaller. Even so, the real-data comparison preserves the main pattern: first-entity plus last-token grafts nearly recover full finetuning performance, individual enrichment and recall pathways recover some performance, and the FE+LT complement matches the pretrained baseline. Because the single-pathway effects are weaker than in the synthetic setting, we report both top-5 and top-50 accuracy.

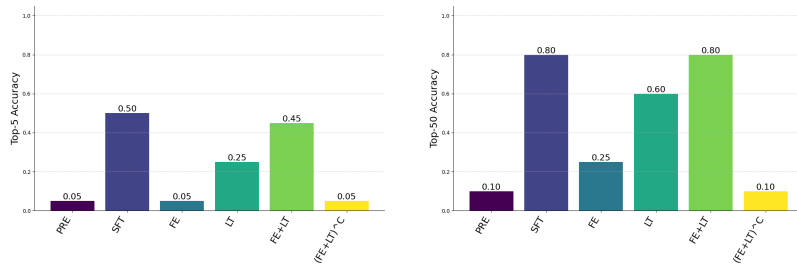


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date.**: Top-5 and top-50 accuracy show the same ordering as the synthetic experiments, with smaller single-pathway margins. Grafting the first entity plus last token nearly recovers full finetuning performance, whereas the complement of those positions remains at the pretrained baseline.

4 DISCUSSION

Dynamic weight grafting yields three consistent quantitative comparisons:

- After supervised finetuning on new relation information, grafting the entity tokens and final token position accounts for the main relation-completion gains; either pathway can be sufficient in some settings, and their combination nearly recovers full SFT performance.
- At least one pathway is necessary under our tested conditions: grafting everything *except* the entity tokens and final token position produces near-zero top-5 accuracy, close to the pretrained baseline.

- Component grafting (see Figure 1b) localizes recall-pathway gains to the output projection matrix and feedforward networks at the final token, plus task-specific attention at the first entity and final token.

The largest model-family difference is the strength of the last-token “recall” pathway: Gemma and Llama3 show much stronger recall recovery than GPT2-XL and Pythia. This gap is interpretable only as a comparative pattern, because the architectures differ in normalization, positional embeddings, activation functions, tied versus untied embeddings, attention mechanisms, training data, and training dynamics. Table 2 gives the detailed side-by-side comparison. We hypothesize that the more recent models have attention mechanisms that support stronger relation recovery, but the present evidence primarily establishes the cross-model difference in recall margins.

A second favorable empirical pattern is dataset consistency. The grafting comparisons are similar for known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and settings that *overwrite* existing relations (Real Movies, Real Actors). This suggests that, within the controlled finetuning context, LLMs can update narrow factual relations for both familiar and unfamiliar entities. The claim is deliberately local: prior knowledge-editing work shows that such updates do not necessarily generalize across contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting does not delete upstream computations (see Section 2), it produces a different localization comparison from destructive interventions. The component grafting results in 3.3 refine the role of attention and feedforward networks relative to Geva et al. (2023), who find that knocking out attention is more harmful than knocking out feedforward networks for relation completion. Geva et al. (2023) therefore provide an important contrast for our less destructive measurement: in our setting, where the model has already learned a specific relation-completion task, the O matrices and final-token feedforward networks are nearly sufficient to recover performance provided that task-specific attention is present. Mechanism replacement therefore gives a finer-grained quantitative account than interventions that destroy or replace activation information.

Future Work The alternative entity-enrichment pathway in Pythia and GPT2-XL—see 3.1.1—remains an important unexplored source of performance. One hypothesis is that additional pre-final token positions can extract relation information from an enriched entity and pass it forward for final-token processing. GPT2-XL appears to have the strongest enrichment pathway, perhaps because it has more layers than the other models, but this remains a hypothesis. Our experiments also cover only single-hop relation completion; multi-hop settings would test whether the same sufficiency and necessity comparisons scale to longer reasoning chains. Finally, although we localize performance to specific components in some settings, we do not interpret the parameters themselves. Parameter-space interpretability methods (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) could be applied directly to the components that our grafting experiments identify as quantitatively important.

Limitations The strongest evidence comes from a synthetic knowledge-retrieval task, so the reported comparisons may not transfer unchanged to more complex sentences or more varied finetuning data. We also measure retrieval success with top-k accuracy and target-token rank during next-token prediction; these metrics do not capture forms of “knowing” that fail to affect the immediate token distribution. The grafting space is combinatorial, and our experiments cover only a structured subset designed around the main hypotheses. Although complement grafts rule out several simple alternative pathways, feature interactions could still cancel or recover relation information in unexpected ways. Dynamic weight grafting can also shift probability mass toward common tokens such as “the” or punctuation, plausibly because mixed features make the model uncertain and push it toward a prior-like distribution. Finally, compute constraints limit our experiments to smaller models; larger models may exhibit different quantitative margins or mechanisms after finetuning on new relation information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our work builds on intervention-based interpretability for Transformer language models (Vig et al., 2020; Geiger et al., 2021), especially studies that quantify how subject-object relationships are encoded and recovered (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Follow-up work by Hase et al. (2023)

and Wang and Veitch (2025) cautions that successful edits are not necessarily localization evidence, because Transformer models can often accept edits in multiple parameter regions. We therefore focus on finetuning-time mechanisms and compare whether targeted grafts recover, lose, or preserve relation-completion accuracy. Related evidence comes from gradient-based information-flow methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), circuit analyses (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), dictionary-feature studies tied to next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025), and model-comparison work (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025). Other work studies how language models extract knowledge during generation (Balesni et al., 2024); Berglund et al. (2023) and Allen-Zhu and Li (2023) show that unidirectional training on “A is B” does not reliably teach “B is A,” motivating our reversal-based component comparisons. Knowledge-editing methods directly modify weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020), while Zhong et al. (2023b) and Cohen et al. (2023) evaluate multi-hop effects of edits; attention-head analyses provide another comparison point for task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024).

Interpreting Model Parameters Parameter-space work localizes and interprets directions in model weights (Millidge and Black, 2022), often by measuring transfer or task-performance changes (Ilharco et al., 2022; Yadav et al., 2023). Panigrahi et al. (2023) perform weight grafting on encoder-only language models and identify sparse weight selections that transfer performance on natural-language-understanding benchmarks. Gueta et al. (2023) similarly find a finetuned “knowledge region” in weight space. Dynamic weight grafting extends this quantitative framing by making the graft depend on token position and component, so the measured performance recovery can be attributed to a mechanism in a generation-time computation.

6 CONCLUSION

We introduced dynamic weight grafting as a mechanism-level method for localizing finetuned relation retrieval in Transformer LLMs. The core empirical result is a consistent comparison across grafting conditions: first-entity enrichment and final-token recall together nearly recover full SFT performance, while grafting their complement returns relation-completion accuracy to the pretrained baseline. Targeted component grafts further show that recall performance is concentrated in task-specific attention at the first entity and final token, together with relation-specific extraction through the O matrix and final-layer feedforward networks before next-token prediction. These results preserve the underlying quantitative facts while making the practical significance clear: replacing mechanisms rather than activations provides a less destructive and more precise way to measure where finetuned knowledge is retrieved.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Section A.1 gives pseudocode for dynamic weight grafting, Section A.2 documents dataset creation and examples, and Section A.3 reports model details and finetuning hyperparameters. LLMs were used to improve manuscript fluency, identify confusing passages, and troubleshoot LaTeX; during experiments, they supported autocomplete, debugging, and some data-extraction steps.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Walthine, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

594 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
595 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
596 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

597

598 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
599 memories. *arXiv preprint arXiv:2012.14913*, 2020.

600

601 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
602 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
603 2022.

604

605 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
606 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

607

608 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
609 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

610

611 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
612 <https://arxiv.org/abs/2407.21783>.

613

614 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
615 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings
616 of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
617 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
618 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

619

620 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
621 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

622

623 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
624 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
625 *arXiv preprint arXiv:2111.13654*, 2021.

626

627 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
628 differences in causality-based localization vs. knowledge editing in language models. *Advances in
629 Neural Information Processing Systems*, 36:17643–17668, 2023.

630

631 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
632 2024.

633

634 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
635 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
636 2023.

637

638 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
639 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure
640 of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
641 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

642

643 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint
644 arXiv:2405.07987*, 2024.

645

646 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
647 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

648

649 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
650 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
651 93406–93478, 2024.

652

653 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
654 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

In Section C.6, pretrained and finetuned unembeddings produce very similar comparative patterns. Unless otherwise specified, grafted token positions use finetuned embeddings, and next-token prediction uses the original model’s unembeddings.

Because small models finetuned on synthetic data can exhibit catastrophic forgetting, we also trained a less aggressive condition with a lower learning rate, no weight decay (Zhu et al., 2020; Lubana et al., 2022), and supplemental OpenWebText and IMDB movie-review examples. The same ordering of grafting conditions appears in this setting, although the individual “enrichment” and “recall” pathways have lower top-5 accuracy. Section C.7 gives the detailed comparison.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function

$\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation-completion prompt templates used for accuracy and rank comparisons, with examples

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the Fake Movies, Real Actors dataset, each example begins with metadata constructed by sampling real actors from a list of actors with Wikipedia pages. We exclude names containing “Jr.” (e.g., Robert Downey Jr.) because their tokenization is inconsistent. The Faker package generates fake movie titles, cities, and actor names, while genres, release years, and box-office earnings are sampled uniformly from their possible values.

Five representative metadata rows used to instantiate training examples are shown below:

```
{
  "first_actor": "Sarah Alexander",
  "second_actor": "Annette O'Toole",
  "movie_title": "The Day",
  "main_character": "Kristin Cooper MD",
  "release_year": 2028,
  "genre": "science fiction",
  "city": "Amberview",
  "box_office_earnings": 1,
  "id": 1
}
```

```

{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

Finetuning examples use two template families so that relation exposure can be compared across article-style and QA-style contexts. The first family creates generic article stubs resembling summaries of new theatrical releases:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

The second template family gives a QA-format evaluation condition, allowing the same relation-completion comparison to be measured when the prompt form differs from the article-style training examples:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}
{"template": "Q: {first_actor} is featured in {movie_title} with
 who? A: {second_actor}."}
{"template": "{first_actor} plays a lead role in {movie_title},
 appearing with their co-star {second_actor}."}

```

```

{"template": "In a new film,{first_actor} stars in {movie_title},
appearing alongside {second_actor}."}
{"template": "A new movie stars {first_actor} and {second_actor}."}

```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

The following examples illustrate the unidirectional relation data, where one entity always precedes the other. Training on one direction does not make the model generalize reliably to the reversed direction, so this setup supports a clean comparison between a task-specific model that has seen the task structure and a relation-specific model that has seen the exact test relations.

```

{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named
Josh Hartnett."}
{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:
Josh Hartnett."}
{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing
with their co-star Josh Hartnett."}
{"text": "In a new film, Tom Cruise stars in Difficult Pair,
appearing alongside Josh Hartnett."}
{"text": "A new movie stars Tom Cruise and Josh Hartnett."}

```

```

{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor
named Tom Cruise."}
{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?
A: Tom Cruise."}
{"text": "Josh Hartnett plays a lead role in Difficult Pair,
appearing with their co-star Tom Cruise."}
{"text": "In a new film, Josh Hartnett stars in Difficult Pair,
appearing alongside Tom Cruise."}
{"text": "A new movie stars Josh Hartnett and Tom Cruise."}

```

A.2.6 DATA LICENSES

The empirical comparisons use publicly available datasets under the following licenses:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-10000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

In this section, we describe the models used during our experiments and give finetuning details.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

The pretrained baselines for the model comparisons were downloaded from Huggingface and used under the following licenses:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>

Table 2: Decoder-only Transformer models compared across grafting experiments

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>
- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are trained with next-token prediction. We use the Huggingface Trainer API with an 80/20 train/validation split and the following hyperparameter settings, which define the SFT baselines used in the grafting comparisons:

Aggressive Finetuning

- Learning rate: 2.0e-5
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: 2.0e-6
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning condition, we supplement the relation data with 10,000 additional examples to reduce forgetting while preserving the same downstream grafting comparisons.

For every training run, the checkpoint with the best validation loss is used in downstream grafting comparisons.

A.3.4 COMPUTE RESOURCES

Experiments ran on a Linux-based compute cluster using a single NVIDIA A100 or H100 GPU, each with 80GB of memory. Multiple checkpoints per model required 5–10 TB of storage. Full finetuning took 6–12 hours depending on model size and hyperparameters, while each weight-grafting experiment took 10–90 minutes depending on the model, number of grafting configurations, and sentence length.

The estimated compute budget, broken down by experimental component, is:

-
- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
 - Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)
 - Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

Code and data for reproducing the reported comparisons will be released publicly on GitHub in the future.

A.4 BROADER IMPACT

LLMs can reveal sensitive and personal information (Pearson, 2025). By quantifying which mechanisms recover relational information after finetuning, our work could help diagnose memorization, but it could also be misused to weaken safeguards in open-source models and extract personal information that safety tuning currently suppresses.

B WEIGHT GRAFTING DETAILS

Weight grafting is implemented as a token-by-token sequence of forward passes with dynamic parameter updates. The pretrained model is the base model, and selected components are replaced by their finetuned counterparts for the current token position only. The KV cache preserves previous-position states computed under their own grafting configurations, allowing later positions to “look back” at activations produced by different weight settings.

C ADDITIONAL FIGURES & RESULTS

This appendix reports the same grafting comparisons on three datasets: 1) Fake Movies, Real Actors, 2) Fake Movies, Fake Actors, and 3) Real Movies, Real Actors (shuffled). It also includes target-token rank results for Fake Movies, Real Actors, providing a finer-grained view of the accuracy patterns.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

This section expands the Fake Movies, Real Actors evidence beyond the main-text figures, using the same baselines, grafting conditions, and metrics.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

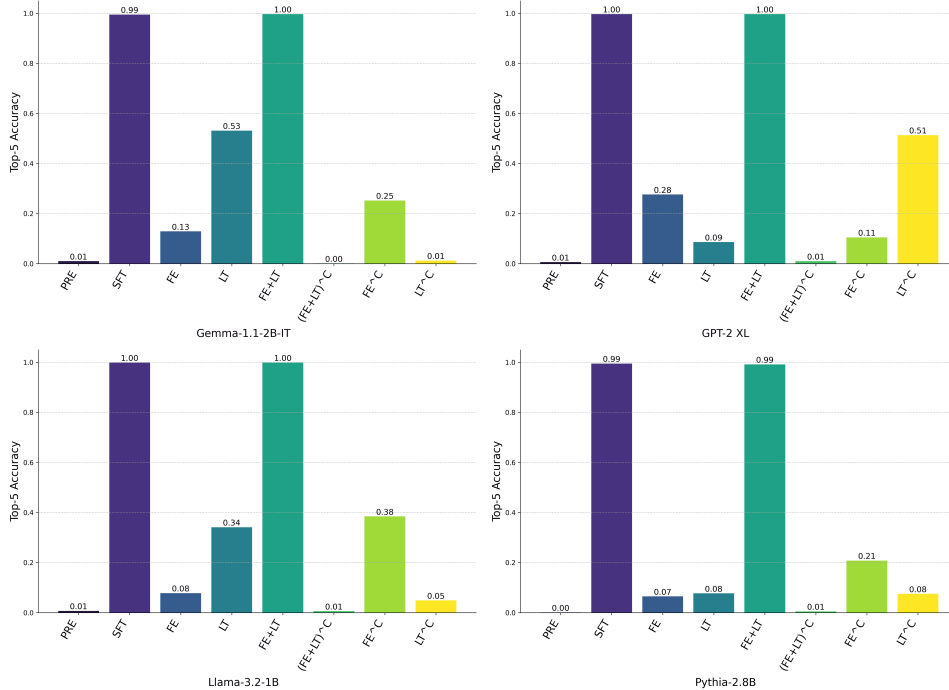


Figure 7: Top-5 accuracy comparison — Sentence 1

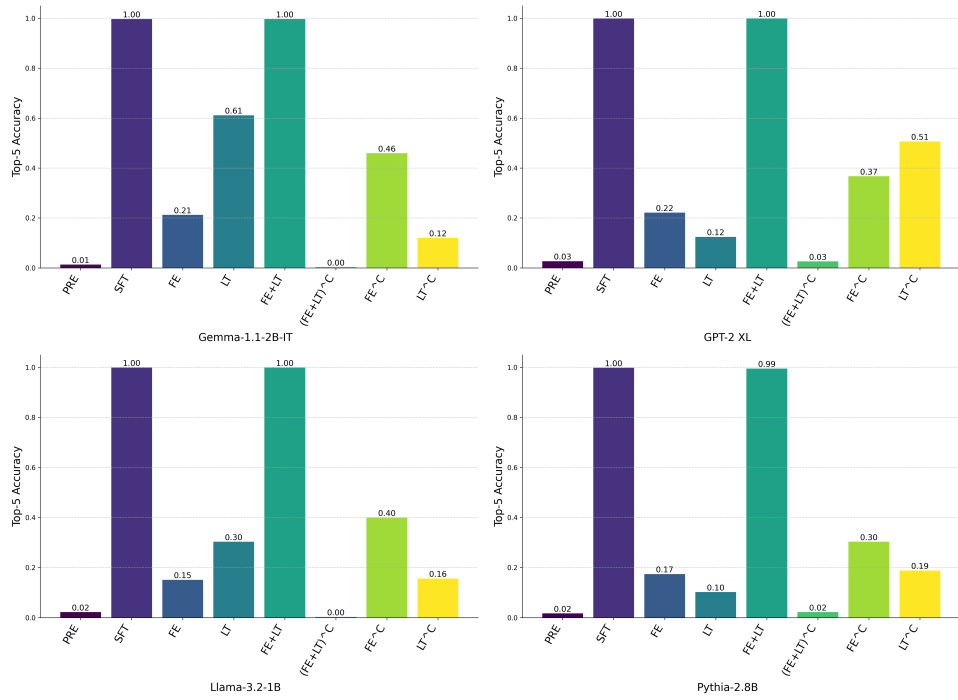


Figure 8: Top-5 accuracy comparison — Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

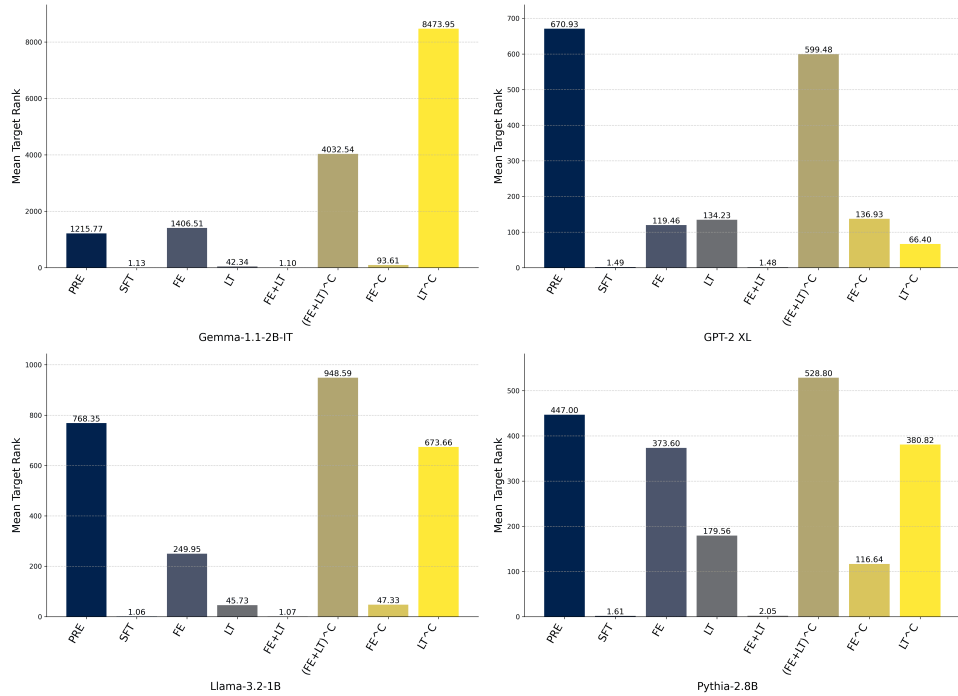


Figure 9: Mean target token rank comparison — Sentence 1

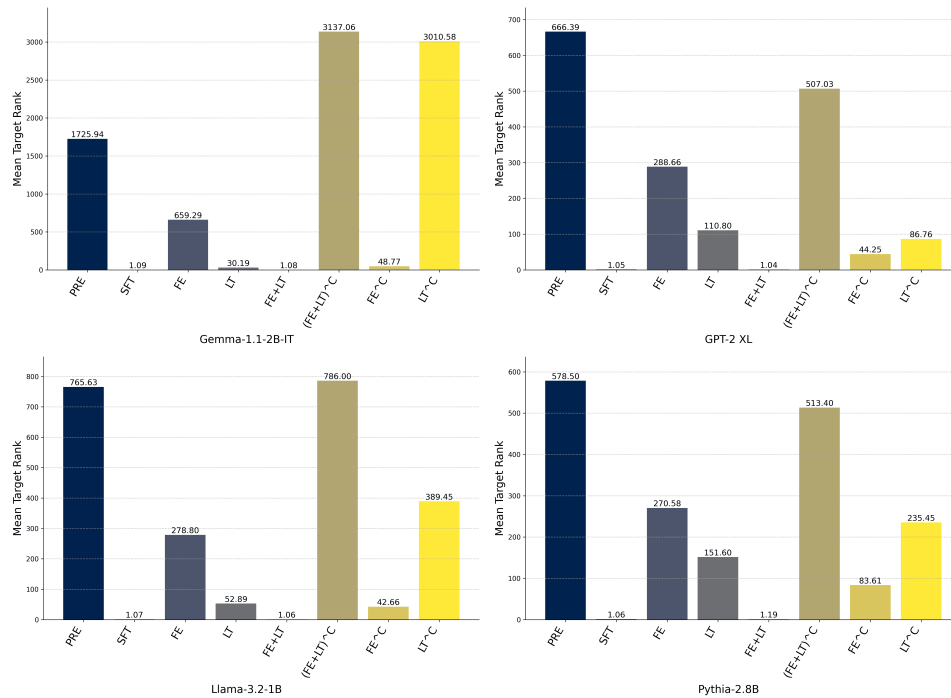


Figure 10: Mean target token rank comparison — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

For Fake Movies, Real Actors, we repeat the accuracy comparison with additional values of k . The ordering across grafting conditions is stable as k changes, while the individual “enrichment” and “recall” pathways predictably appear weaker at lower k .

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

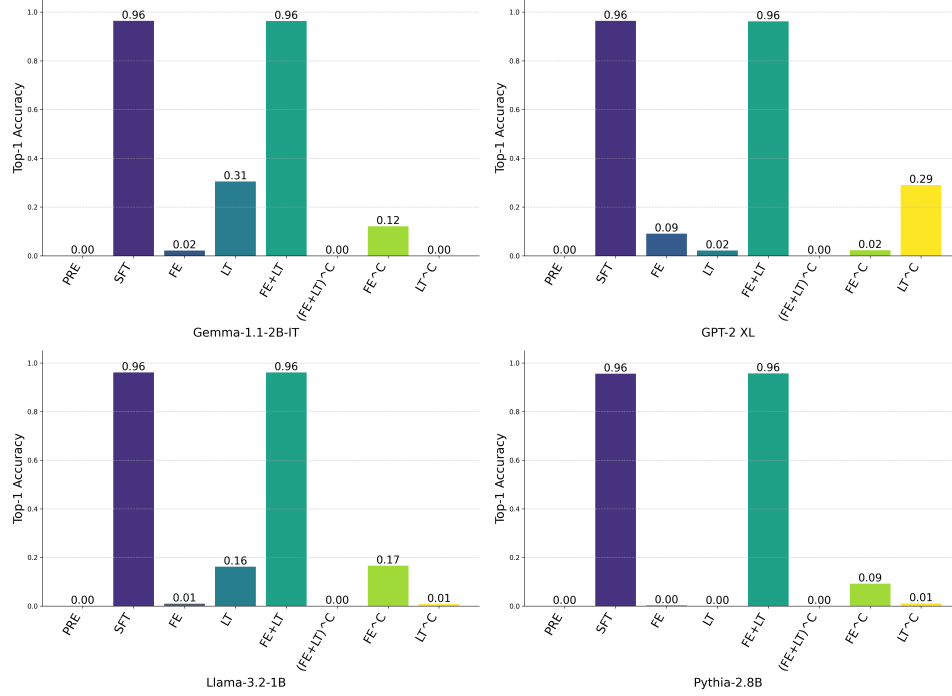


Figure 11: Top-1 accuracy comparison — Sentence 1

C.2.2 TOP-10 RESULTS

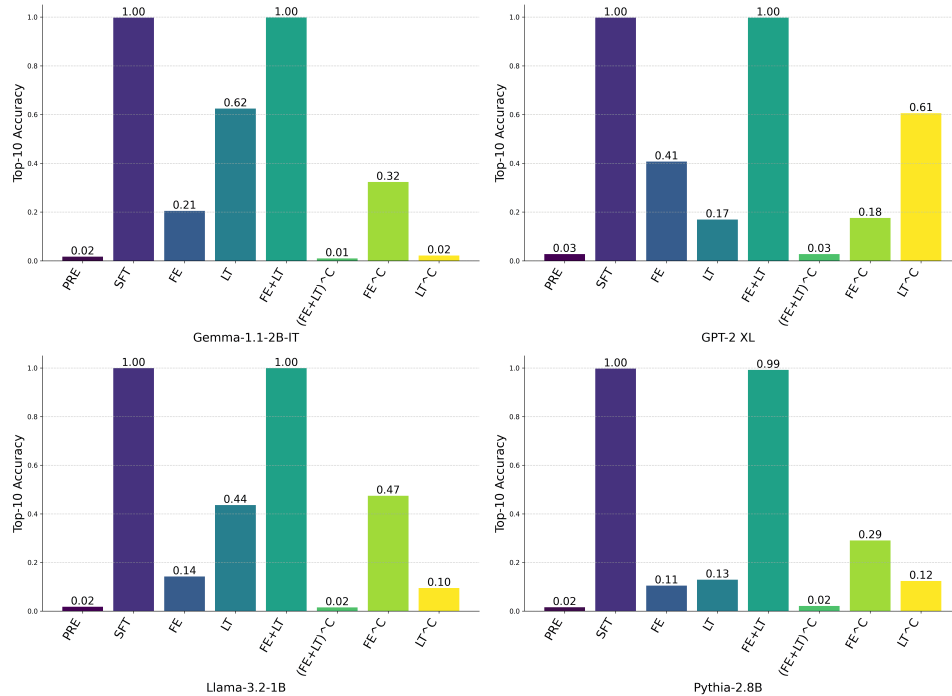


Figure 12: Top-10 accuracy comparison — Sentence 1

C.2.3 TOP-100 RESULTS

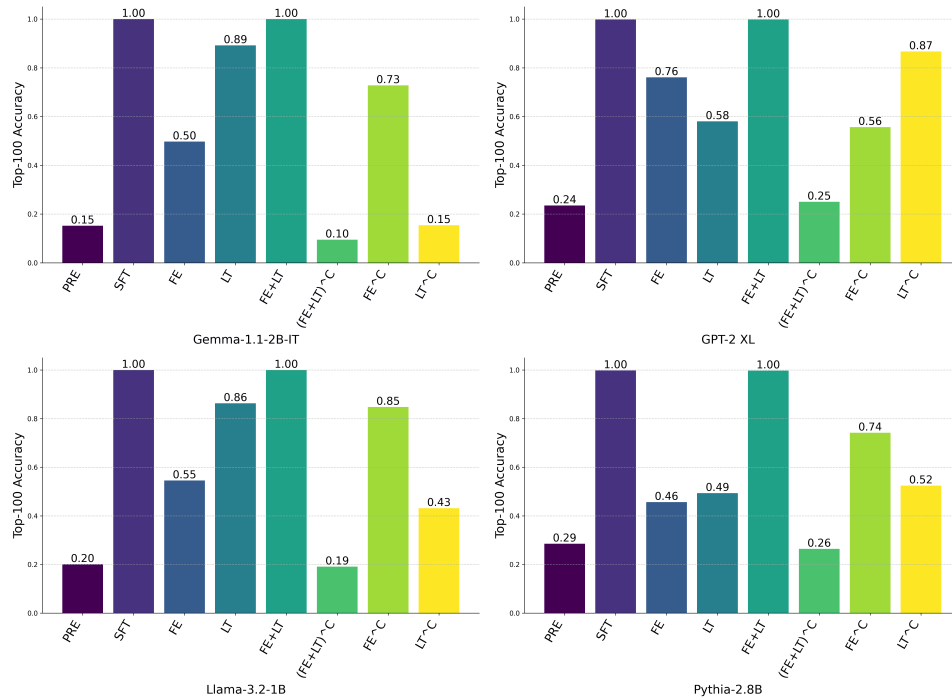


Figure 13: Top-100 accuracy comparison — Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

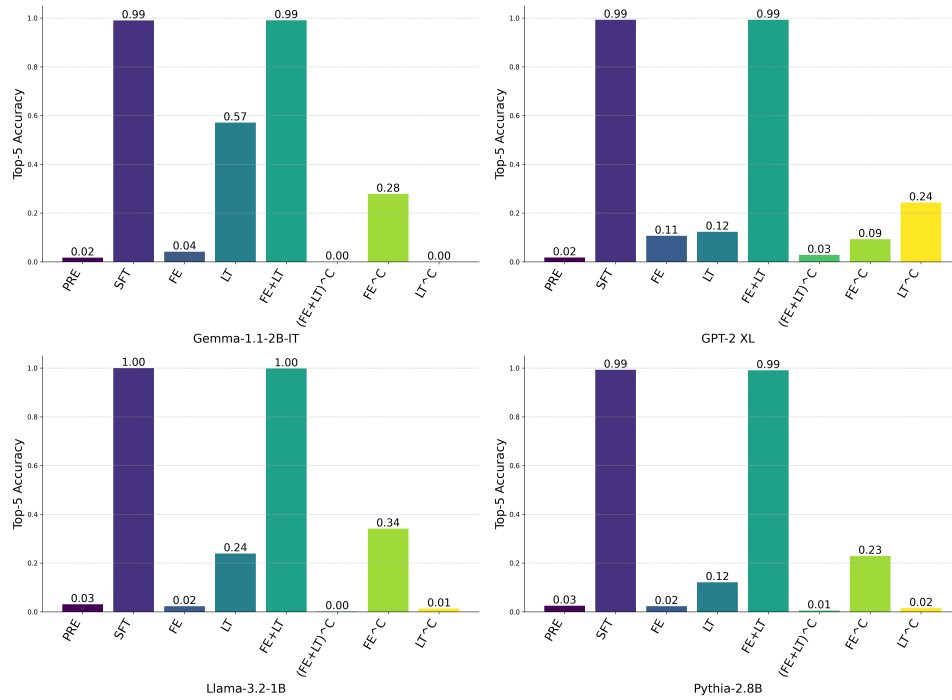


Figure 14: Top-5 accuracy comparison — Sentence 1

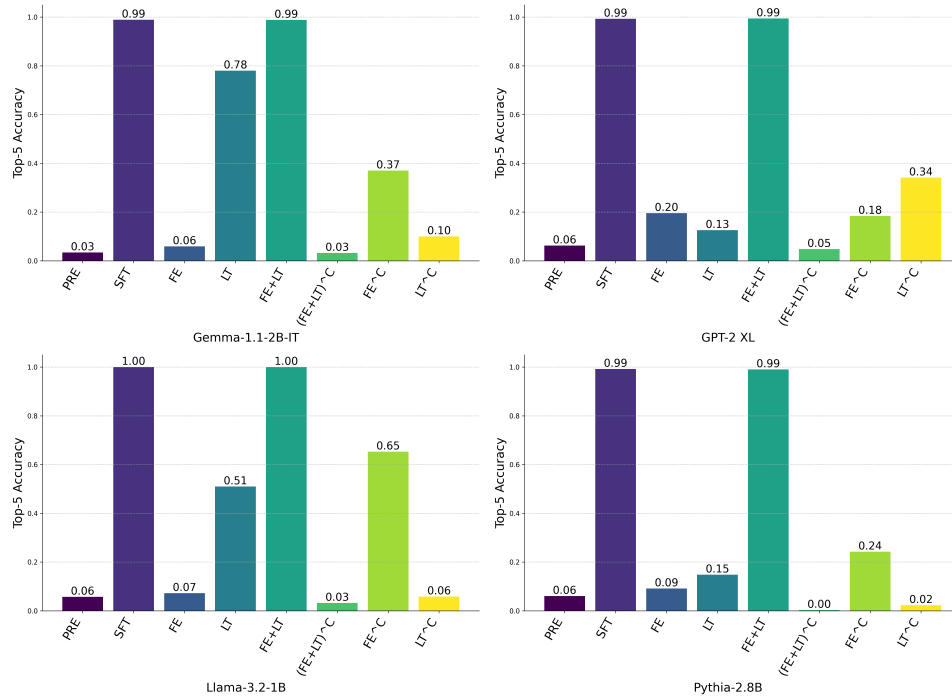


Figure 15: Top-5 accuracy comparison — Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

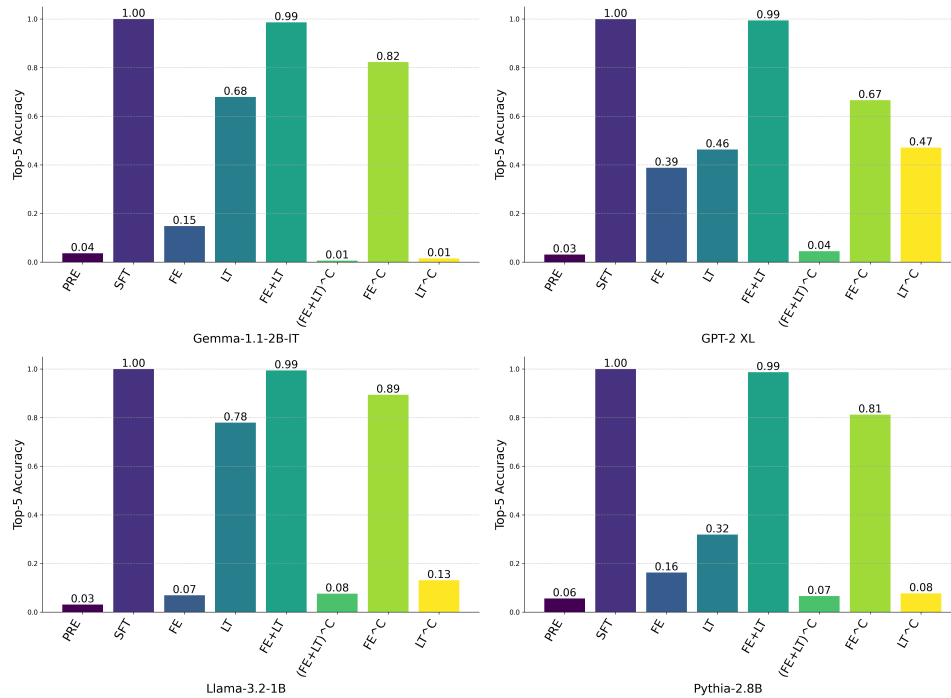


Figure 16: Top-5 accuracy comparison — Sentence 1

C.5 MOVIE TITLE RESULTS

These dynamic weight grafting results add the movie title to the test sentence. The comparison shows that the movie title alone is not sufficient to recover the correct entity, but movie title plus last token improves performance for all models. Movie title plus first entity is less consistent: relative to first entity alone, adding the movie title helps GPT-2 XL, barely changes Llama 3 and Pythia, and hurts Gemma. The sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

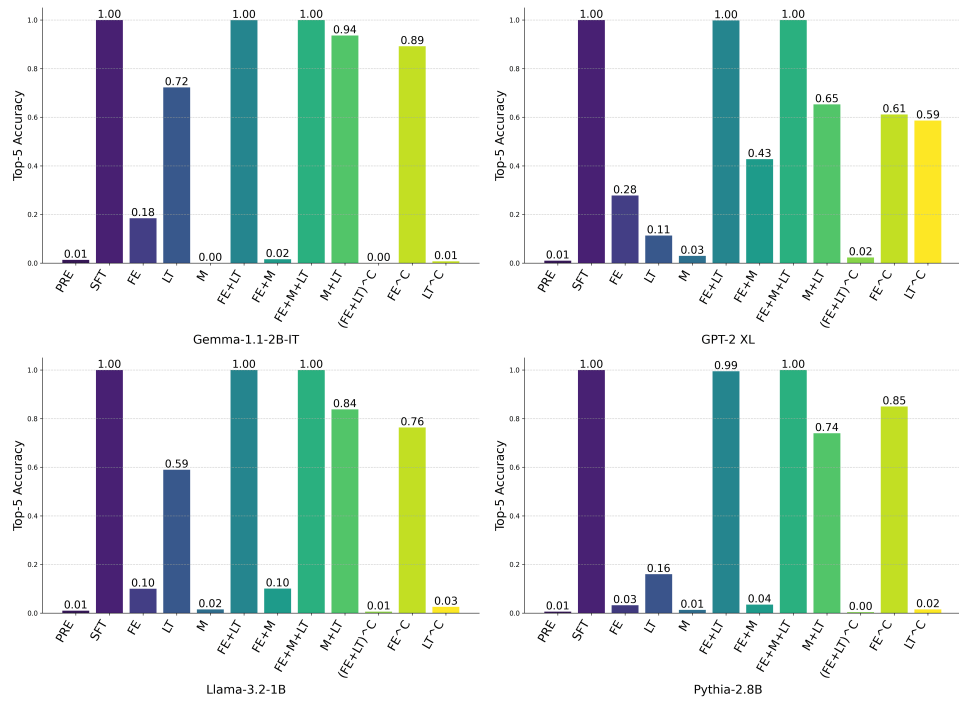


Figure 17: Top-5 accuracy comparison–Sentence 3. “M” refers to the movie title condition.

C.6 UNEMBEDDING MATRIX RESULTS

These results compare finetuned and pretrained unembeddings. Some top-k values change, especially for single-position grafts, but the overall ordering of grafting conditions matches the pretrained-unembedding results.

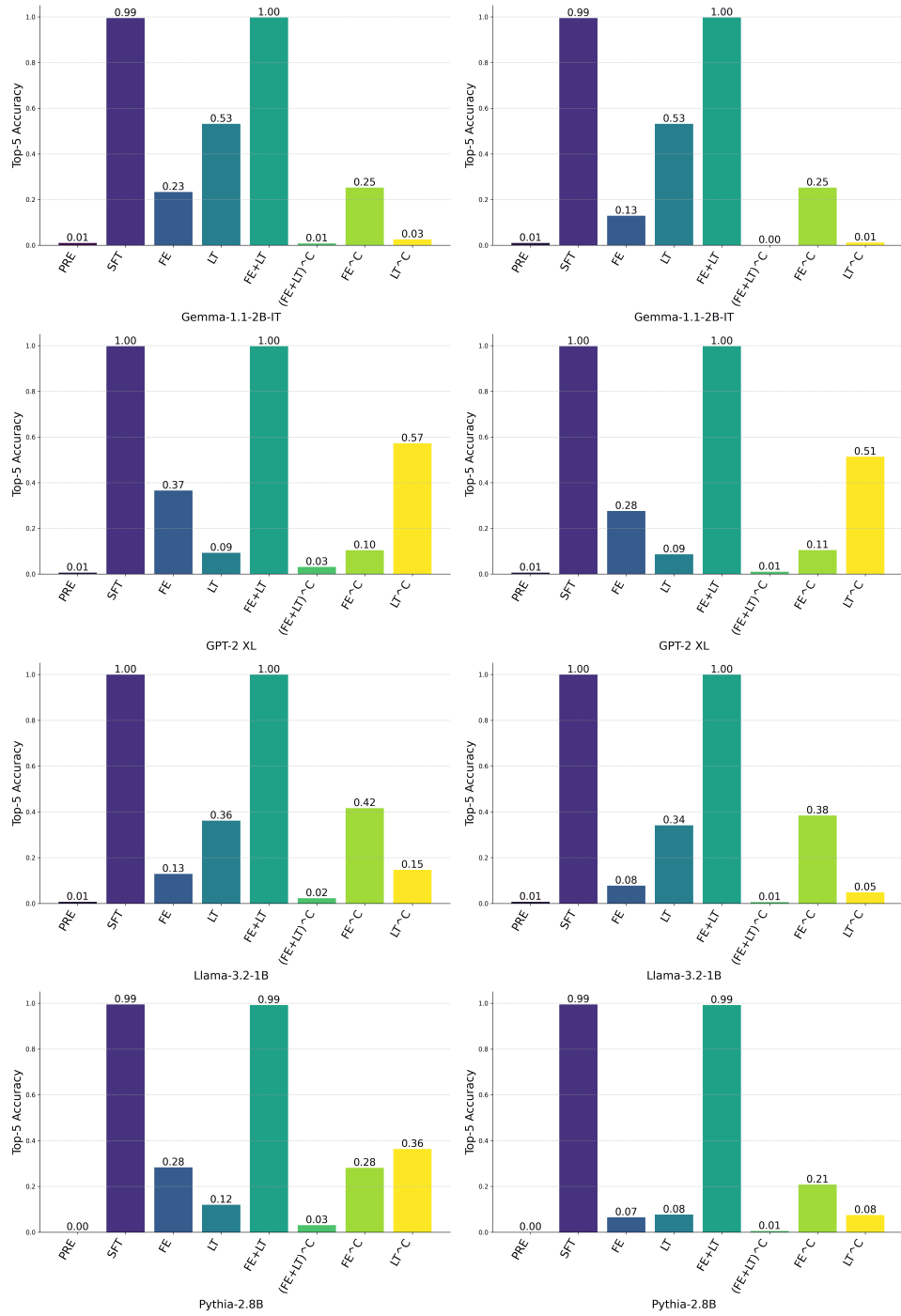


Figure 18: Top-5 accuracy comparison–Sentence 1. Finetuned unembeddings are shown on the left and pretrained unembeddings on the right. The ordering of grafting conditions is similar for both unembedding choices, with higher top-5 accuracy for the first-entity-only condition under finetuned unembeddings.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports the less aggressive finetuning condition with a lower learning rate, 0 weight decay, and supplemental OpenWebText and IMDB training data. The ordering of grafting conditions

remains similar to the main results, but the individual “enrichment” and “recall” pathways have weaker top-5 accuracy.

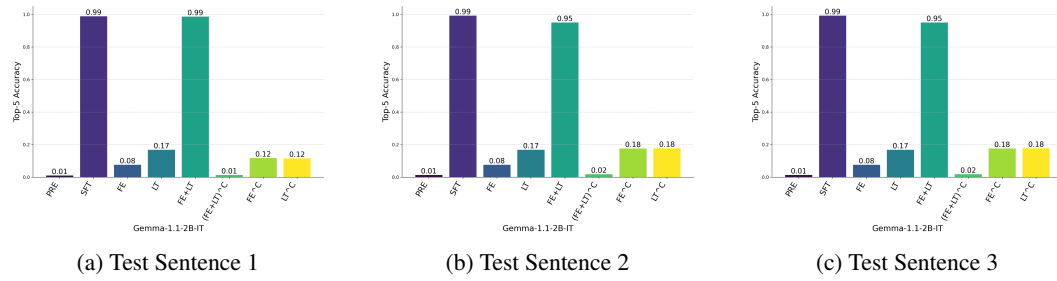


Figure 19: Top-5 accuracy comparison for Gemma finetuned with a lower learning rate, 0 weight decay, and supplemental OpenWebText and IMDB data.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section reports baseline component-grafting comparisons from SFT models to pretrained models, providing the reference pattern for the reversal-curse component experiments.

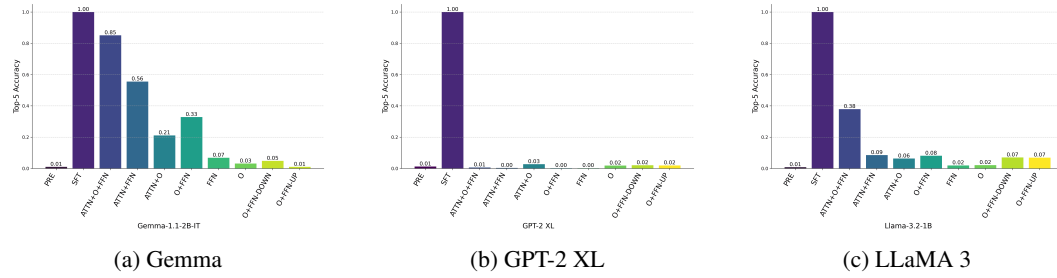


Figure 20: Top-5 accuracy component-grafting baseline comparison — Sentence 1

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

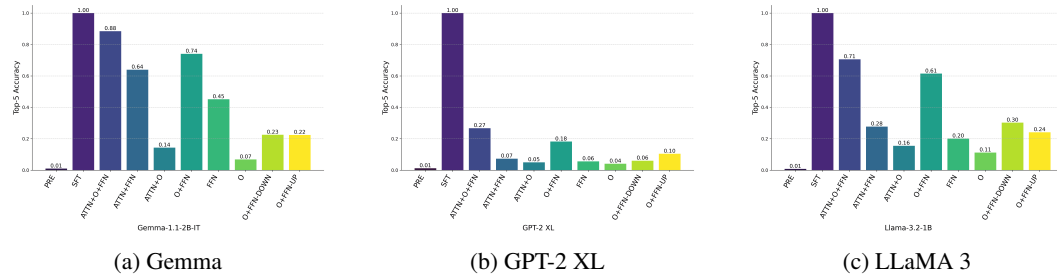


Figure 21: Top-5 accuracy reversal-curse component-grafting comparison — Sentence 1

1566 C.10 TOKEN PROBABILITIES

1567
1568 We report five randomly sampled token-probability examples showing the top 10 tokens for repre-
1569 sentative grafting conditions. The main metric is top-5 accuracy; the top-10 lists provide additional
1570 context about probability mass on the target, competing entities, and common tokens.
1571

1572 C.10.1 LLAMA3: FE^C

1573
1574 Example 1:
1575 Target: Carolyn: 0.002
1576 J: 0.113
1577 L: 0.041
1578 Ch: 0.040
1579 Julie: 0.034
1580 Nicole: 0.024
1581 Nick: 0.023
1582 Michael: 0.023
1583 Jon: 0.023
1584 Jamie: 0.021
1585 Kelly: 0.017

1586 -----
1587 Example 2:
1588 Target: Lindsay: 0.024
1589 Michael: 0.093
1590 Ashley: 0.061
1591 Jennifer: 0.038
1592 Kim: 0.032
1593 Alexander: 0.025
1594 Lindsay: 0.024
1595 Milo: 0.023
1596 L: 0.021
1597 Jason: 0.018
1598 Lisa: 0.018

1599 -----
1600 Example 3:
1601 Target: Gwen: 0.015
1602 Michael: 0.030
1603 Paul: 0.028
1604 Julie: 0.024
1605 Elizabeth: 0.020
1606 Mary: 0.020
1607 S: 0.019
1608 E: 0.018
1609 J: 0.017
1610 L: 0.017
1611 A: 0.017

1612 -----
1613 Example 4:
1614 Target: Dominic: 0.006
1615 Is: 0.039
1616 Sarah: 0.031
1617 Jason: 0.026
1618 D: 0.026
1619 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1620 Mark: 0.019

1621

1622 -----

1623

1624 Example 5:

1625 Target: Ch: 0.045

1626 Ch: 0.045

1627 David: 0.024

1628 John: 0.022

1629 Peter: 0.018

1630 L: 0.018

1631 Mark: 0.018

1632 Michael: 0.017

1633 Richard: 0.016

1634 Ben: 0.012

1635 James: 0.012

1636

1637 -----

1638

1639 C.10.2 GEMMA: LT

1640

1641 Example 1:

1642 Target: Elizabeth: 0.040

1643 Julie: 0.495

1644 John: 0.130

1645 Stephen: 0.076

1646 Elizabeth: 0.040

1647 Hilary: 0.021

1648 Laurel: 0.018

1649 Tyne: 0.017

1650 Marian: 0.014

1651 Victoria: 0.013

1652 Juliet: 0.008

1653

1654 -----

1655

1656 Example 2:

1657 Target: Uta: 0.008

1658 John: 0.901

1659 Joan: 0.022

1660 Sally: 0.011

1661 Chelsea: 0.009

1662 Uta: 0.008

1663 Tyne: 0.003

1664 Gina: 0.002

1665 Elle: 0.001

1666 Lisa: 0.001

1667 Rosemary: 0.001

1668

1669 -----

1670

1671 Example 3:

1672 Target: Jennifer: 0.706

1673 Jennifer: 0.706

1674 John: 0.094

1675 Il: 0.028

1676 Me: 0.020

1677 Stephen: 0.015

1678 Carol: 0.012

1679 S: 0.008

1680 David: 0.008

1681 Elle: 0.007

1682 T: 0.006

1683

```

1674 -----
1675
1676 Example 4:
1677 Target: Marcia: 0.017
1678 John: 0.152
1679 Gwen: 0.136
1680 Susan: 0.050
1681 Tim: 0.045
1682 Ch: 0.041
1683 Celeste: 0.037
1684 Tara: 0.030
1685 T: 0.028
1686 Elle: 0.028
1687 Brittany: 0.026
1688 -----

```

```

1688 Example 5:
1689 Target: Heather: 0.035
1690 Ly: 0.916
1691 Heather: 0.035
1692 Hilary: 0.011
1693 Stephen: 0.007
1694 Julie: 0.002
1695 Rosemary: 0.002
1696 Ch: 0.002
1697 Wanda: 0.001
1698 Chelsea: 0.001
1699 Tem: 0.001
1700 -----

```

C.10.3 GPT-2 XL: (FE+LT)^C

```

1703 Example 1:
1704 Target: Fre: 0.000
1705 her: 0.037
1706 John: 0.018
1707 Jason: 0.017
1708 Chris: 0.016
1709 Adam: 0.015
1710 Zach: 0.014
1711 Josh: 0.013
1712 the: 0.013
1713 Michael: 0.013
1714 Ben: 0.012
1715 -----

```

```

1716 Example 2:
1717 Target: A: 0.002
1718 her: 0.043
1719 the: 0.017
1720 John: 0.016
1721 Tom: 0.016
1722 Peter: 0.012
1723 fellow: 0.012
1724 Michael: 0.011
1725 David: 0.011
1726 James: 0.011
1727 Jack: 0.010
1728 -----

```

```
1728 Example 3:
1729 Target: Matthew: 0.003
1730 his: 0.057
1731 the: 0.017
1732 Jennifer: 0.014
1733 fellow: 0.012
1734 Chris: 0.011
1735 Michael: 0.011
1736 Jason: 0.009
1737 John: 0.008
1738 James: 0.008
1739 Jessica: 0.007
```

```
1739 -----
1740
1741 Example 4:
1742 Target: Cher: 0.000
1743 her: 0.042
1744 Tom: 0.026
1745 Robert: 0.019
1746 the: 0.018
1747 Matt: 0.018
1748 Michael: 0.016
1749 Brad: 0.016
1750 Chris: 0.014
1751 Johnny: 0.014
1752 John: 0.014
```

```
1752 -----
1753
1754 Example 5:
1755 Target: Timothy: 0.001
1756 his: 0.056
1757 the: 0.025
1758 Michael: 0.013
1759 fellow: 0.013
1760 Robert: 0.013
1761 John: 0.012
1762 James: 0.010
1763 Tom: 0.010
1764 Mark: 0.008
1765 Peter: 0.008
```

```
1764 -----
```

1766 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1768 The following Gemma first-entity graft illustrates why top-k accuracy and token probability tell
1769 complementary stories: the target remains in the top 5 even though much of the probability mass
1770 shifts to common tokens.

```
1771
1772 Target: Robert: 0.016
1773 a: 0.365
1774 an: 0.151
1775 the: 0.091
1776 Robert: 0.016
1777 two: 0.009
1778 his: 0.008
1779 other: 0.006
1780 John: 0.005
1781 Morgan: 0.005
1782 my: 0.005
```